Paper Critique: Resource-Efficient Transformer Pruning for Finetuning of Large Models

Full Name: Mathis DOUTRE
Student ID: 904-04-8015
Paper #: 6
Title : Resource-Efficient Transformer Pruning for Finetuning of Large Models

---

1 Problems Adressed

Ilhan et al. (2024) address the high GPU-memory cost of fine-tuning large transformers. State-of-the-art models (e.g., ViTs, BERT, GPT) contain billions of parameters, enabling strong generalization but making fine-tuning memory-intensive. For instance, a 70B-parameter LLaMA-2 needs 14 GB for inference and 70 GB for 16-bit fine-tuning. These ressources are out of reach for many users.

To reduce memory load, many efficient fine-tuning techniques have been explored, but each has limits. **Quantization** reduces model size but is not task-aware and may lose information, especially with outlier weights [2, 3]. Quantization is typically applied post-training during inference, so they do not reduce fine-tuning memory.

Adapter-based methods train small modules while freezing the backbone. For example, **LoRA** adds low-rank adapters [4], and prefix-tuning learns task-specific prefix tokens [5]. However, these methods still require loading the full backbone on GPU during training, leaving the memory bottleneck unsolved for very large models.

Model pruning is another approach: **Pruning** removes weights not needed for the target task and has been effective for shrinking models and speeding inference. However, it creates a trade-off. **Pruning before the adaptation** (*Pre-fine-tuning pruning*) risks discarding weights needed to learn the task, often reducing downstream accuracy [6, 7]. **Pruning after the fine-tuning** (*Post-fine-tuning*) better preserves accuracy but still requires loading and updating the entire model during the initial fine-tuning. Indeed, prior research found pruning gives the best results when done after fine-tuning.

In summary, the paper addresses two main problems:

Training memory challenge: Fine-tuning transformers often exceeds the memory of GPUs. The goal is to reduce memory during training without sacrificing accuracy.

Limitations of existing methods: Prior approaches either compromise task performance or still require holding the full model in memory. The paper targets this gap by enabling resource-aware pruning during fine-tuning to maintain accuracy while lowering memory cost.

2 New Idea and Strengths

The authors introduce RECAP (REsource and ComputAtion-efficient Pruning) [1], a method that integrates pruning into the fine-tuning process to lower GPU memory requirements while maintaining accuracy. The key idea is to bridge the gap between pre- and post-finetune pruning by performing an iterative fine-tuning that only ever operates on a subset of the model at any given time. This approach is illustrated in the Figure 1, which contrasts RECAP against conventional strategies: pure **pre-finetune pruning** (which can lead to "suboptimal pruning" by itself) and pure **post-finetune pruning** (which faces a "resource bottleneck" of needing the full model in GPU memory). **RECAP's design** uses both CPU and GPU resources in tandem to avoid those pitfalls.

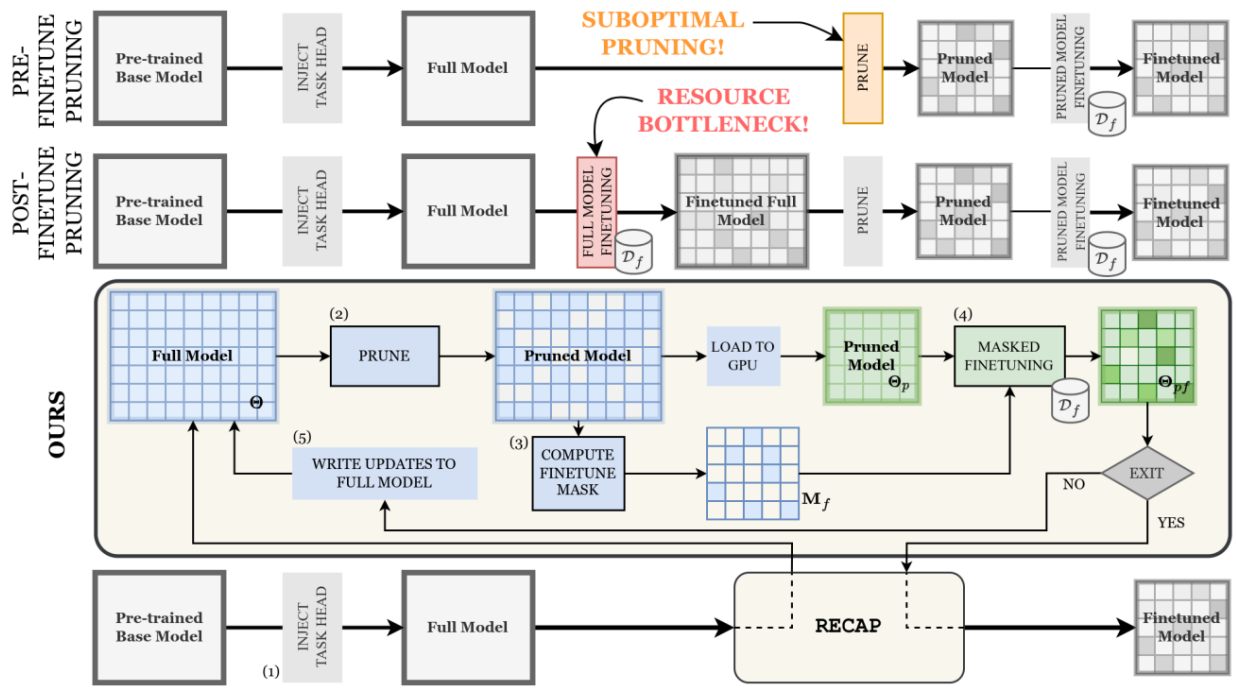

Figure 1: System architecture of RECAP. Components in blue are done on CPU, and those in green are done on GPU (from Ilhan et al., 2024 [1]).

How RECAP works:

RECAP runs an iterative three-stage pipeline (pruning, fine-tuning, updating), repeated until the model is sufficiently tuned.

Stage 1, Pruning (CPU): The method processes one model chunk at a time: the full model stays in CPU memory, and only the active chunk is evaluated for pruning for each iteration. Weights are organized into groups (e.g., entire attention heads or MLP neurons). Group importance is scored with a first-order Taylor criterion using gradients, estimating the loss increase if the group were removed. Groups are ranked by this importance metric, the algorithm **prunes** away the **least important** weight groups. For example, RECAP can drop entire attention heads or hidden units with low-impact. This stage runs on CPU only and outputs a pruned chunk for the next step.

Stage 2, Fine-tuning (on GPU, with masked updates): During this training phase, only the pruned smaller model is on the GPU. RECAP then performs *Fisher-informed masked updates*: it trains a selected **subset of parameters** (higher estimated impact from gradients) while freezing the rest.
This has **two benefits**: *(i) Memory savings:* by updating only (for example) 25% of the remaining weights, the GPU only needs to allocate gradient and optimizer tensors for that fraction, significantly cutting memory overhead. *(ii) Training stability:* the model prevents the pruned regions from quickly regaining importance by not adjusting every weight, making pruning more stable.

Stage 3, Updating (CPU coordination): After fine-tuning the pruned smaller model, the GPU weights are transfered into the CPU model. RECAP then repeats the *prune* $\rightarrow$ *load* $\rightarrow$ *fine-tune* $\rightarrow$ *update* cycle for a certain number of rounds or until a stopping criterion is met.

This process covers the model without ever loading it fully on the GPU. The result is a model pruned to the target sparsity and adapted to the task, achieved within GPU memory limits.

Strengths and innovations of RECAP:

At any given time, only a **fraction of the weights** are on the GPU, and only **a fraction** of those weights are **updated. Memory overhead drops** across weights, activations, gradients, and optimizer states. The authors report up to **65%** lower GPU memory than existing pruning or adapter methods. In ViT fine-tuning, RECAP uses roughly **one-third** of the memory of full fine-tuning or post-pruning fine-tuning. Figure 2 shows RECAP on the top-left of the accuracy–memory curve: it can exceed other methods at the same memory budget or match their accuracy with far less memory. In contrast, approaches like Post-FT pruning or Movement pruning still load the entire model on GPU, so their memory matches full fine-tuning.

Despite aggressive pruning and partial updates, RECAP **maintains accuracy close to full-model fine-tuning**. On CIFAR-100 classification and Cityscapes/KITTI segmentation, RECAP reaches similar mIoU while using 30–40% less GPU memory. On GLUE with BERT-base, it matches full fine-tuning and exceeds naive pre-fine-tuning pruning at the same sparsity.

RECAP is task-agnostic and model-agnostic: it groups transformer weights into structured

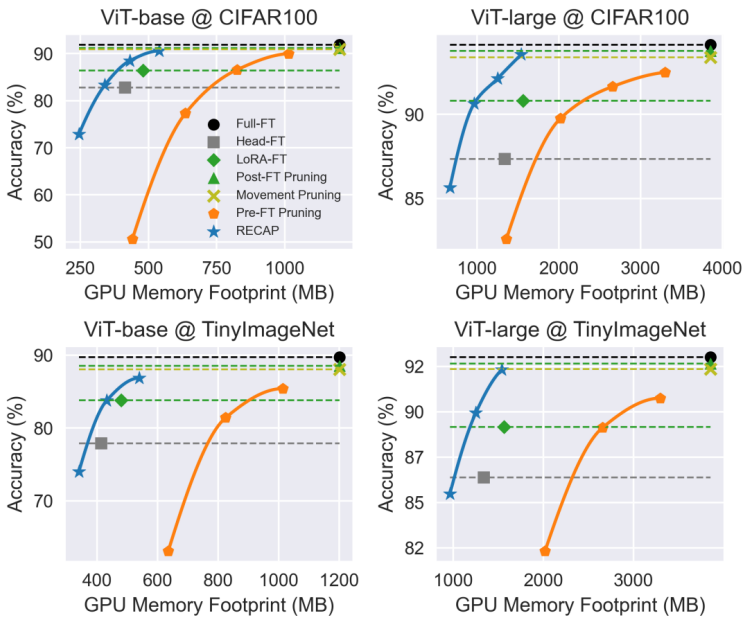

Figure 2: Accuracy vs. GPU memory for ViT-Base/Large: RECAP matches or exceeds baselines with equal memory, or achieves similar accuracy with less memory; post–fine-tuning and movement pruning peak at Full-FT memory (from Ilhan et al. [1]).

units and requires no task-specific architecture changes. Evaluations on computer vision (ViTs for classification and segmentation) and NLP (BERT on GLUE), indicate broad applicability. The same procedure should extends to other transformer models, including larger language models, making it a good option for resource-constrained fine-tuning.

Unlike pipelines that assume a full-model fine-tune before pruning, **RECAP does not require full-model fine-tune**. We can start from a pretrained model and apply RECAP on limited hardware. This enables users with modest GPUs to fine-tune large models without loading the entire model at once.

3 Weaknesses and Suggestions

RECAP introduces **additional steps** (iterative rune, mask, fine and tune) compared to standard fine-tuning; this **increase total training time**.

As the Figure 3 shows RECAP reduces GPU memory but takes longer than full FT, reflecting a **trade-off** of time for memory. Future work could optimize the pipeline by overlapping CPU computations with GPU training.

RECAP assumes the **full model resides in CPU memory** when it is not on the GPU. While RAM is usually larger and cheaper than GPU VRAM, very large models can still exceed available host memory (e.g., we need 140 GB just to store a 70B-parameter model in FP16). The paper mainly evaluates ViT-Large and BERT-base–scale models (hundreds of millions of parameters), so practical scalability to larger tens-of-billions-parameter models remains unproven.

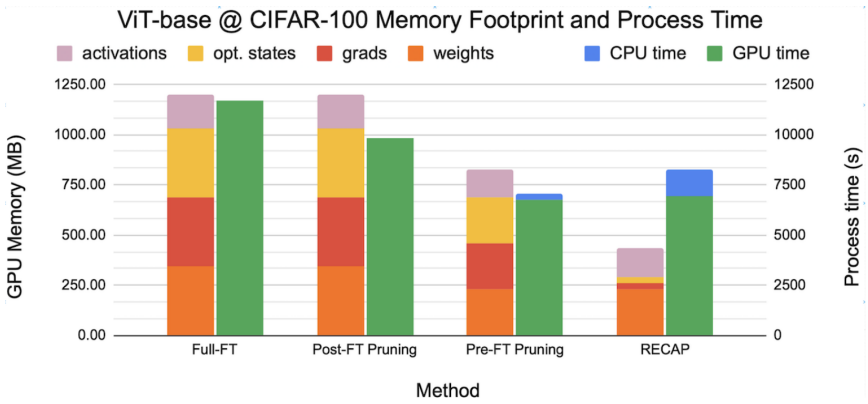

Figure 3: Breakdown of GPU memory footprint (left bars) and process time (right bars) for ViT-base at CIFAR100 (from Ilhan et al. [1]).

RECAP saves memory mainly by reducing on-GPU weights and limiting gradient storage, but in tasks with large inputs or high-resolution features, **activations can dominate**. In the paper's segmentation study (Mask2Former on Cityscapes), activations accounted for **57% of total memory**. An adapter-based approach used **less memory** by freezing the backbone and skipping its gradients, at the cost of 0.5–1 mIoU lower accuracy. This suggests RECAP's advantage **narrows** when activation size is the primary constraint, not model size. This limits its applicability to high-resolution tasks like semantic segmentation or video. Future work could include freezing the most activation-heavy layers, to control memory use when activations dominate.

Because **pruning reduces capacity**, RECAP can fall a little behind full fine-tuning. On Cityscapes, a RECAP-pruned *Swin-Large* reaches **79.5 mIoU versus 82.7 for full fine-tuning**. This is because the partial updates may miss interactions that benefit from joint optimization of all weights. **Suggestions:** We could run a brief fine-tune at the end if we have access to the necessary ressources. This assumes the RECAP-pruned model is close enough to a good optimum that full fine-tuning can resume from it under the same conditions.

The method is **designed for transformer-based models**, with pruning structured around heads and feed-forward units. This covers many modern architectures, but its applicability to other types like CNNs is unexplored. While transformers dominate many domains, this limitation is worth being noted. **Suggestions:** Future work could test RECAP's ideas on other architectures, possibly requiring new grouping strategies. Applying them to larger transformer-based models (e.g., GPT-scale) would help clarify scalability and the issues from longer sequence lengths and higher activation memory.

References

[1] Fatih Ilhan, Gong Su, Selim Furkan Tekin, Tiansheng Huang, Sihao Hu, and Ling Liu. Resource-efficient transformer pruning for finetuning of large models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. URL https://openaccess.thecvf.com/content/CVPR2024/papers/Ilhan_Resource-Efficient_Transformer_Pruning_for_Finetuning_of_Large_Models_CVPR_2024_paper.pdf.

[2] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS) 2022*, 2022. URL https://arXiv.org/abs/2208.07339.

[3] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022. URL https://arXiv.org/abs/2210.17323.

[4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021. URL https://arxiv.org/abs/2106.09685.

[5] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190*, 2021. URL https://arxiv.org/abs/2101.00190.

[6] Trevor Gale, Erich Elsen, and Sara Hooker. The state of sparsity in deep neural networks. *arXiv preprint arXiv:1902.09574*, 2019. URL https://arXiv.org/abs/1902.09574.

[7] Michael Zhu and Suyog Gupta. To prune, or not to prune: Exploring the efficacy of pruning for model compression. 2017. URL https://arXiv.org/abs/1710.01878.